

Feature Alignment and Uniformity for Test Time Adaptation

Shuai Wang¹, Daoan Zhang², Zipei Yan³, Jianguo Zhang², Rui Li¹

¹Tsinghua University

²Southern University of Science and Technology

³Hongkong Polytechnic University

Group 14:

Sambit Sahoo – 22277

Prakash Kumbhakar – 22241

Taashif Bashir – 22347

Motivation and Problem Setting

- ▶ Deep learning performs well when training and test data come from the same distribution.
- ▶ **Real-world challenge:** Domain shift between source (training) and target (test) data causes performance drop.
- ▶ **Test-Time Adaptation (TTA):**
 - ▶ Model adapts during test phase using **unlabeled** target data.
 - ▶ No modification to training or extra supervision.
- ▶ **Goal:** Improve representation quality in the target domain via test-time adaptation.

Representation Revision Perspective

- ▶ Source-trained model generates suboptimal representations for target data.
- ▶ We frame TTA as a **representation revision** task.
- ▶ **Two key properties:**
 - ▶ **Alignment:** Similar images should have similar representations.
 - ▶ **Uniformity:** Representations from different classes should be well-separated and uniformly distributed.
- ▶ Existing works focus on only one aspect; we address **both alignment and uniformity**.

Paper's Approach

▶ Test-Time Feature Uniformity:

- ▶ Use memory bank of past representations/logits.
- ▶ Align model predictions with prototype-based predictions.
- ▶ Achieved via **unsupervised self-distillation (TSD)**.

▶ Test-Time Feature Alignment:

- ▶ Use K-nearest neighbors in memory.
- ▶ Encourage consistency in features and predictions.
- ▶ Introduced as **Memorized Spatial Local Clustering (MSLC)**.

▶ Filtering:

- ▶ **Entropy Filter:** Removes uncertain samples.
- ▶ **Consistency Filter:** Ensures prototype and model predictions agree.

Contributions and Impact

- ▶ New perspective on TTA via feature **alignment and uniformity**.
- ▶ Two strategies: **Test-Time Self-Distillation** (TSD) and **MSLC**.
- ▶ Use of **entropy** and **consistency filters** to mitigate label noise.
- ▶ Achieves **state-of-the-art** results on:
 - ▶ Domain generalization benchmarks: PACS, VLCS, OfficeHome, DomainNet.
 - ▶ Cross-domain medical image segmentation tasks.

Method Overview

- ▶ **Setting:** Pretrained source model $g = h \circ f$ trained on source domain D_s .
- ▶ **Goal:** Adapt model using only **unlabeled target data** $x_i \in D_t$ at test time.
- ▶ **Output:** $p_i = g(x_i) \in \mathbb{R}^C$, where C is the number of classes.
- ▶ **Self-Distillation:**
 - ▶ Get embedding $z_i = f(x_i)$ and logits $p_i = h(z_i)$.
 - ▶ Generate pseudo-label $\hat{y}_i = \arg \max p_i$.
 - ▶ Store (z_i, p_i) in memory bank $\mathcal{B}$.
- ▶ **Prototypes:** For class k :

$$c_k = \frac{\sum_i z_i \cdot \mathbb{I}[\hat{y}_i = k]}{\sum_i \mathbb{I}[\hat{y}_i = k]}$$

Test-Time Self-Distillation

- **Entropy Filter:** Remove top- M uncertain samples with highest entropy:

$$H(p_i) = - \sum \sigma(p_i) \log \sigma(p_i)$$

- **Prototype-based prediction:**

$$y_i^k = \frac{\exp(\text{sim}(z_i, c_k))}{\sum_{k'} \exp(\text{sim}(z_i, c_{k'}))}$$

- **Loss:** Soft label self-distillation loss:

$$L_i(p_i, y_i) = -\sigma(p_i) \log y_i$$

- **Consistency Filter:** Keep only samples where both classifiers agree:

$$M_i = \mathbb{I}[\arg \max p_i = \arg \max y_i]$$

- **Final TSD Loss:**

$$L_{\text{tsd}} = \frac{\sum_i L_i \cdot M_i}{\sum_i M_i}$$

Memorized Spatial Local Clustering

► **Spatial Local Clustering:**

- Features of the same class should align in the latent space, but domain gaps cause misalignment in TTA.
- We use K -nearest neighborhood features from memory to align the target features and reduce noisy labels.

► **Logit Alignment:**

- Align logits of target sample x with K -nearest features in memory.
- Cosine similarity is used to measure the similarity between the target and its nearest neighbors.

► **Loss Function:**

$$L_{\text{mslc}} = \frac{1}{K} \sum_{j=1}^K \text{sim}(z, z_j) \cdot (\sigma(p) - \sigma(p_j))^2$$

Final Objective and Online Adaptation

► Final Objective:

- Combine the two loss functions: TSD and MSLC.
- The final loss function is:

$$L = L_{\text{tsd}} + \lambda L_{\text{mslc}}$$

- λ is the trade-off parameter that balances the losses.

► Online Adaptation:

- For each new sample x_T , predict $p_T = g(x_T)$.
- Perform one-step gradient update using L .
- Adaptation continues as long as test data is available.

Experimental Setup

Datasets:

- ▶ **PACS:** 9,991 examples, 7 classes, 4 domains (art, cartoons, photos, sketches).
- ▶ **OfficeHome:** 15,588 images, 65 classes, 4 domains (art, clipart, product, real).
- ▶ **VLCS:** 10,729 images, 5 classes, 4 domains (Caltech101, LabelMe, SUN09, VOC2007).
- ▶ **DomainNet:** 586,575 images, 345 classes, 6 domains (clipart, infograph, painting, quickdraw, real, sketch).

Models:

- ▶ Evaluated on ResNet-18/50 with Batch Normalization.
- ▶ Also tested on ViT-B/16, ResNeXt-50(32x4d), EfficientNet(B4), and MLP-Mixer(Mixer-L16).

Implementation Details

Source Training:

- ▶ One domain as target, others as source domains.
- ▶ Split source domain data into 80% for training, 20% for validation.
- ▶ Adam optimizer with learning rate $5e^{-5}$.
- ▶ All models initialized with ImageNet-1K pre-trained weights (except ViT-B/16 and MLP-Mixer).

Test-Time Adaptation:

- ▶ Adam optimizer, batch size = 128.
- ▶ Trade-off parameter $\lambda = 0.1$.
- ▶ All trainable layers updated.
- ▶ PyTorch for implementation.

Ablation Study & Hyperparameter Tuning

Ablation Study – PACS, ResNet-50

Variant	Acc. (%)
ERM (Baseline)	84.59
+ TSD	87.80
+ Entropy Filter	88.40
+ Consistency Filter	88.92
+ MSLC (Full)	89.41

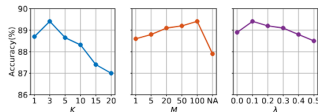


Figure 3. Sensitivity analysis about the number of nearest neighbors K in memorized spatial local clustering, entropy filter hyperparameter M and trade-off parameter λ (cf. Eq. 7).

Hyperparameter Sensitivity

- ▶ **K (Neighbors):** 3–5 best; high K → noise
- ▶ **M (Entropy):** Stable across range
- ▶ λ : Optimal at 0.1

Accuracy vs. K , M , λ

Each module improves results. Parameters are interpretable and robust.

Efficiency Analysis

- ▶ Maintains memory bank $\rightarrow$ extra GPU cost at test-time.
- ▶ **Default:** Batch size 128 $\rightarrow$ 89.4% accuracy, 14GB memory.
- ▶ **Optimized:** Batch size 64 $\rightarrow$ 89.1%, 7.85GB, fastest runtime.
- ▶ Trade-off: Accuracy vs. memory vs. speed.
- ▶ **Affine-only BN update:** 89% accuracy, 2GB saved memory.

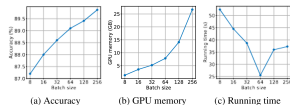


Figure 5. Computational complexity with different batch sizes at test time adaptation. From left to right: accuracy, GPU memory and running time on four domains of PACS [30] dataset.

Accuracy, memory runtime vs.
batch size (PACS)

Comparative Study

Test-Time Adaptation (TTA) Methods:

- ▶ Compared with ERM, BN, Tent, PL, SHOT-IM, T3A, ETA, LAME.
- ▶ Results on ResNet-18 (Table 1):
 - ▶ **Paper's Method:** 87.32 on PACS, 64.83 on OfficeHome, 73.61 on VLCS, 40.19 on DomainNet.
- ▶ Results on ResNet-50 (Table 2):
 - ▶ **Paper's Method:** 89.41 on PACS, 68.67 on OfficeHome, 74.52 on VLCS, 47.73 on DomainNet.

Limitations

- ▶ **Limited to high-level tasks:** Our method is not validated on low-level vision tasks like:
 - ▶ Image dehazing
 - ▶ Denoising
 - ▶ Super-resolution

These tasks lack semantic prototypes and entropy measures, limiting method applicability.

- ▶ **Hyperparameter tuning:** Performed on training domain validation set, which may not generalize well to test domains.
- ▶ **Future direction:** Explore better test-time hyperparameter tuning strategies to improve robustness in unseen domains.

DiffRect: Latent Diffusion Label Rectification for Semi-supervised Medical Image Segmentation

- ▶ **Task 1:** Run on ACDC Dataset
- ▶ **Task 2:** Run on Decathlon Prostate Dataset
- ▶ **Task 3:** Run on Synapse Dataset

DiffRect: Latent Diffusion Label Rectification

DiffRect Overview:

- ▶ Semi-supervised medical image segmentation using latent diffusion models.
- ▶ It takes a small amount of labeled data and helps the model use it to predict labels for the larger, unlabeled data.

Key Components:

- ▶ **Label Propagation:** Aligns sparse labels with latent representations.
- ▶ **Latent Diffusion:** Refines label information in a probabilistic latent space.

Impact:

- ▶ Achieves high segmentation accuracy with limited labeled data.

Evaluation Metrics for Segmentation Results

Four Key Metrics:

- ▶ **Dice Coefficient:** Measures overlap between predicted and ground truth segmentation. $\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$.
- ▶ **HD95 (Hausdorff Distance at 95%):** Maximum distance between predicted and ground truth contours.
- ▶ **Jaccard Index:** Similarity between predicted and ground truth. $\text{Jaccard} = \frac{|A \cap B|}{|A \cup B|}$.
- ▶ **ASD (Average Symmetric Surface Distance):** Average distance between predicted and ground truth surfaces.

Task 1: ACDC Dataset - Overview

ACDC Dataset:

- ▶ The **ACDC dataset** is a publicly available dataset for cardiac MRI segmentation, containing 150 multi-equipments CMRI recordings with fully annotated regions for left ventricle, myocardium, and right ventricle.
- ▶ **Number of Classes:** 3 (Left Ventricle, Myocardium, Right Ventricle).

Task 1: ACDC Dataset - Results

Evaluation Metrics:

- ▶ Dice, Jaccard, HD95 and ASD were used to evaluate the segmentation performance.

Results per Class :

Class 1:[0.873, 0.789, 3.499, 0.837]

Class 2:[0.881, 0.790, 1.473, 0.628]

Class 3:[0.919, 0.858, 2.339, 0.679]

Average Results:[0.891, 0.812, 2.437, 0.715]

Task 2: Decathlon Prostate Dataset - Overview

Decathlon Prostate Dataset:

- ▶ The Decathlon Prostate dataset is part of the Medical Decathlon competition and includes prostate MRI scans for the task of prostate segmentation.
- ▶ **Number of Classes:** 3 (Liver, Prostate, Background).
- ▶ **Total Samples:** 634 MRI scans used for segmentation tasks.
- ▶ **Class Details:**
 - ▶ Class 1 Peripheral Zone (PZ).
 - ▶ Class 2 Transition Zone (PZ)
 - ▶ Class 3 (Background): Everything in the image that is not part of the liver or prostate.

Task 2: Decathlon Prostate Dataset - Results

Evaluation Metrics:

- Dice, Jaccard, HD95 and ASD were used to evaluate the segmentation performance.

Results per Class:

Class 1:[0.538, 0.388, 13.419, 4.272]

Class 2:[0.766, 0.629, 14.563, 3.987]

Class 3:[0.000, 0.000, 50.000, 10.000]

Average Results:

[0.652, 0.508, 13.991, 4.129]

Task 3: Synapse Dataset - Overview

Synapse Dataset:

- ▶ The Synapse dataset consists of multi-organ CT scans for the task of organ segmentation.
- ▶ **Number of Classes:** 9 classes including:
 - ▶ Background (Class 0)
 - ▶ Brain (Class 1)
 - ▶ Left Hippocampus (Class 2)
 - ▶ Right Hippocampus (Class 3)
 - ▶ Left Amygdala (Class 4)
 - ▶ Right Amygdala (Class 5)
 - ▶ Left Thalamus (Class 6)
 - ▶ Right Thalamus (Class 7)
 - ▶ Left Cerebellum (Class 8)
- ▶ **Total Samples:** 2211 samples present in dataset.

Task 3: Synapse Dataset - Progress

- ▶ The training data was initially in `.npz` format, which we converted to `.h5` format.
- ▶ Used `acdc_data_processing.py` for pre-processing and handling data.
- ▶ Created train, test, and validation splits for the Synapse dataset.
- ▶ Modified the code for Synapse dataset handling:
 - ▶ Added specific handling for Synapse data by updating dataset references:
 - ▶ `elif 'Synapse' in dataset:`
 - ▶ Defined reference dictionary:

```
ref_dict = {"1" : 63, "3" : 110, "7" : 222}
```

- ▶ Added code to handle 9 classes in the dataset:
 - ▶ Updated the model and processing pipeline to account for all 9 classes.
 - ▶ Ensured that the Synapse dataset's color mapping and processing reflect the 9 classes.

Thank You!

Questions or Discussion?

We appreciate your time and attention.